O C P

Open / Closed Principle

Open to extend, closed to modify

DEFINITION

Open for extension, closed for modification — add new behavior by adding code, not editing what already works.

"Closed for modification" means the source of a working module stays untouched; "open for extension" means new behavior arrives as a new implementation behind a stable abstraction. You cannot be closed against every possible change — so you predict the one axis that actually varies (a new shape, a new payment method) and put a polymorphic seam there. Everywhere else you stay simple and concrete.

WHY IT MATTERS

Editing tested code to bolt on one more case risks regressing everything that already relied on it, and it turns every feature into a diff in the same hot file — painful reviews, merge conflicts, fragile releases. A seam turns "edit and pray" into "add a class": the new variant cannot reach into the old ones, so existing behavior and its tests stay green while the system grows.

— How it works

Abstraction	The stable contract clients depend on (an interface or abstract base). It names the axis of change without committing to any one variant.
Extension	A new implementation of the abstraction. Adding behavior means adding one of these — never editing the others.
Closed client	The code that uses the abstraction (a calculator, a dispatcher). Written once against the contract and never reopened for a new variant.

HOW TO APPLY

1. Find the axis of change — the thing you keep adding cases for (shapes, channels, formats).
2. Name a stable abstraction over it: an interface with the one method clients need.
3. Move each existing case into its own implementation of that abstraction.
4. Point the client at the abstraction; add the next variant as a new class, touching nothing else.

LITMUS TEST

Can you add the next variant without editing an existing class or switch? If every new case means touching the same file, the seam is missing.

— Smell → fix

Every new shape forces another edit to the same switch:

```
function area(shape) {
switch (shape.kind) {
case 'circle': return pi * shape.r * shape.r
case 'square': return shape.s * shape.s
}
} // every new shape edits THIS function
```

↓ EXTRACT A POLYMORPHIC SEAM

```
interface Shape { area(): number }
class Circle implements Shape { area() { return pi*this.r*this.r } }
class Square implements Shape { area() { return this.s*this.s } }
// new shape = new class; area() callers never change
```

The caller now depends on the Shape abstraction. A new shape is a new file — existing classes and their tests stay closed.

USE WHEN

- ✓ A switch grows a new branch every feature
- ✓ Plugin / variation points

AVOID WHEN

- ▲ Speculative abstraction before a second variant exists (YAGNI)

— Trade-offs & pitfalls

PROS

+ New cases can't break old ones
+ Cleaner diffs and reviews

CONS

− Indirection
− Costly if the abstraction is guessed wrong

COMMON PITFALLS

▲ Guessing the axis wrong — a speculative abstraction you never extend is pure indirection and cost (YAGNI).
▲ Abstracting after the first case instead of the second: wait for a real variant before paying for the seam (rule of three).
▲ A "strategy" with a switch still inside it — the conditional only moved; the seam is not really polymorphic.

WHERE YOU SEE IT

- A payments module with a PaymentProvider interface — Stripe, PayPal and a new provider plug in without touching checkout.
- Angular DI swapping an abstract service token for a different implementation per environment.
- An array of validator objects the form runs in turn — a new rule is one more entry, not an edited loop.

SEE ALSO

Strategy	the canonical OCP mechanism — swap behavior behind an interface
DIP	point the dependency at the abstraction so extensions plug in
Protected Variations	GRASP's name for shielding clients from a predicted change point
LSP	extensions must honor the base contract, or OCP breaks at runtime

Interview & sources

Who coined OCP, and how did the meaning shift?

Bertrand Meyer (1988) meant inheritance-based extension; Robert Martin reframed it around polymorphic interfaces — extend by implementing an abstraction.

Doesn’t OCP conflict with YAGNI?

In tension. Don’t add the seam speculatively; introduce it when the second variant actually appears (the rule of three).

You can’t be closed against every change — so?

Predict the one axis of change from the domain and be open there; accept you will still edit for variations you did not foresee.

OCP vs the Strategy pattern?

Strategy is one concrete way to achieve OCP: put the varying algorithm behind an interface and inject it. OCP is the goal; Strategy, polymorphism and plugins are mechanisms that reach it.

REMEMBER

Add new behavior by adding new code, not by editing code that already works.

SOURCES

Bertrand Meyer — "Object-Oriented Software Construction" (1988): the original open/closed formulation

Robert C. Martin — "Agile Software Development, Principles, Patterns, and Practices" (2003): the polymorphic reinterpretation